

Aether — Plugin System

Extensible Runtime Plugin Architecture

Extend Aether with custom runtimes via the JSON-RPC IPC protocol. Drop a manifest into `~/.aether/plugins/` and your runtime becomes a first-class citizen.

JSON-RPC IPC • Auto-Discovery • Custom Runtimes • Plugin Registry

Plugin Architecture

Aether's plugin system communicates with external binaries via a JSON-RPC style protocol over stdin/stdout. Each plugin is a standalone executable that implements the Runtime trait operations.



JSON-RPC IPC Protocol

Aether spawns the plugin binary as a child process, sends a JSON request to stdin, and reads the JSON response from stdout. A 60-second timeout prevents hung plugins from blocking operations.



Standalone Binaries

Plugins are compiled executables in any language. They receive a JSON request, process it, and write a JSON response. No SDK required — just implement the protocol.



Manifest-Driven

Each plugin is described by a JSON manifest file that declares its name, version, runtime kind, binary path, and supported capabilities.



Full Runtime Trait

Plugins implement the same operations as built-in runtimes: build, run, stop, status, delete, and list. They participate in the Decision Engine and migration system.

Aether CLI



JSON Request (stdin)



Plugin Binary



JSON Response (stdout)

Plugin Manifest

The manifest file describes the plugin to Aether. It is a simple JSON file placed in `~/.aether/plugins/`.

Manifest Structure

```
{
  "name": "firecracker",
  "version": "1.0.0",
  "runtime_kind": "firecracker",
  "command": "/usr/local/bin/aether-firecracker",
  "capabilities": [
    "build",
    "run",
    "stop",
    "status",
    "delete",
    "list"
  ]
}
```

Manifest Fields

Field	Type	Description
name	string	Human-readable plugin name and registry key
version	string	Semantic version of the plugin
runtime_kind	string	Custom runtime identifier (used in workload specs)
command	string	Absolute path to the plugin binary
capabilities	string[]	Operations the plugin supports

File Location

Plugin manifests are JSON files in `~/.aether/plugins/`. Aether scans this directory during discovery, loading every `*.json` file as a potential manifest. Invalid manifests are logged as warnings and skipped.

Protocol Messages

Request	Payload	Response	Payload
BuildRequest	spec_json	BuildResponse	image_json
RunRequest	image_json, spec_json	RunResponse	instance_json
StopRequest	instance_json	StopResponse	success (bool)
StatusRequest	instance_json	StatusResponse	status_json
DeleteRequest	instance_json	DeleteResponse	success (bool)
ListRequest	(none)	ListResponse	instances_json

Plugin Lifecycle

From discovery to removal, Aether manages the full plugin lifecycle through CLI commands.

1 Discover

Scan `~/.aether/plugins/` for `*.json` manifest files. Valid manifests are merged into the plugin registry, keyed by name.

2 Register

Each discovered manifest is registered in the persistent plugin registry (`plugins.json`). The binary path is validated to ensure the executable exists.

3 List

View all registered plugins with their versions, runtime kinds, and capabilities. Plugins are available for workload deployment immediately after registration.

4 Remove

Unregister a plugin by name. The binary is not deleted — only the registry entry is removed. Re-run discover to re-register.

CLI Commands

```
# Discover plugins from ~/.aether/plugins/
$ aether plugin discover
Discovered 2 plugins:
  firecracker v1.0.0 (6 capabilities)
  wasm-edge   v0.3.0 (4 capabilities)

# List registered plugins
$ aether plugin list
NAME          VERSION  RUNTIME    CAPABILITIES
firecracker   1.0.0    firecracker build,run,stop,status,delete,list
wasm-edge     0.3.0    wasm       run,stop,status,list

# Remove a plugin
$ aether plugin remove wasm-edge
Removed plugin: wasm-edge
```

Using a Plugin Runtime

```
# Workload spec targeting a plugin runtime
apiVersion: aether/v1
kind: Workload
metadata:
  name: microvm-app
spec:
  runtime:
    preferred: firecracker
    allow: [firecracker]
  requirements:
    cpu: "2"
    memory: 512Mi
    storage: 5Gi
  build:
    context: .
    dockerfile: Dockerfile
    registry: registry.example.com

# Deploy using the plugin
$ aether run workload.yaml
Runtime: firecracker (plugin)
Deploying microvm-app ... done
```

Writing a Custom Plugin

Implement the JSON-RPC protocol in any language. Here is a minimal example in Python.

aether-my-runtime (Python example)

```
#!/usr/bin/env python3
"""Minimal Aether runtime plugin implementing the JSON-RPC IPC protocol."""

import json
import sys

def handle_request(request):
    msg_type = request.get("type")

    if msg_type == "BuildRequest":
        spec = json.loads(request["spec_json"])
        image = {"name": spec["metadata"]["name"], "tag": "latest",
                "digest": None, "runtime": "my-runtime"}
        return {"type": "BuildResponse", "image_json": json.dumps(image)}

    elif msg_type == "RunRequest":
        image = json.loads(request["image_json"])
        instance = {"id": f"{image['name']}-001", "name": image["name"],
                   "runtime": "my-runtime", "state": "Running"}
        return {"type": "RunResponse", "instance_json": json.dumps(instance)}

    elif msg_type == "StatusRequest":
        status = {"state": "Running", "ready": True,
                 "message": None, "restart_count": 0}
        return {"type": "StatusResponse", "status_json": json.dumps(status)}

    elif msg_type == "StopRequest":
        return {"type": "StopResponse", "success": True}

    elif msg_type == "DeleteRequest":
        return {"type": "DeleteResponse", "success": True}

    elif msg_type == "ListRequest":
        return {"type": "ListResponse", "instances_json": "[]"}

    return {"error": f"Unknown request type: {msg_type}"}

if __name__ == "__main__":
    request = json.loads(sys.stdin.readline())
    response = handle_request(request)
    print(json.dumps(response))
```

Installation Steps

Write binary



Create manifest.json



Place in ~/.aether/plugins/



aether plugin discover

The plugin protocol is intentionally simple: read JSON from stdin, write JSON to stdout. This makes it possible to write plugins in any language — Rust, Go, Python, Bash, or even a shell script.

Extend **Aether**

Add any runtime to Aether's universal control plane with a manifest and a binary.

Quick Plugin Setup

```
# 1. Create the manifest
$ cat > ~/.aether/plugins/my-runtime.json <<'EOF'
{
  "name": "my-runtime",
  "version": "1.0.0",
  "runtime_kind": "my-runtime",
  "command": "/usr/local/bin/aether-my-runtime",
  "capabilities": ["build", "run", "stop", "status", "delete", "list"]
}
EOF

# 2. Discover and register
$ aether plugin discover
Discovered 1 plugin: my-runtime v1.0.0

# 3. Deploy workloads to your custom runtime
$ aether run workload.yaml
Runtime: my-runtime (plugin)
Deploying my-app ... done
```

6

Protocol Messages

JSON

Wire Format

60s

IPC Timeout

Any

Language

Your Runtime, Aether's Control Plane

Firecracker microVMs, WebAssembly runtimes, custom hypervisors — if it can run workloads, Aether can manage it. Write a plugin, drop it in, and deploy with the same CLI used for Podman, Kubernetes, KubeVirt, and Metal3.